

WatchUp

Contrato de implementação — aplicativo Android nativo

Versão 1.0 (derivada do protótipo) **Plataforma** Android (Kotlin)

Fonte Protótipo + brief + handoff **Escopo** Todas as telas do MVP

Este documento converte o protótipo clicável do WatchUp em requisitos de implementação. Cada regra, campo e estado descrito abaixo é **rastreável ao protótipo** — a coluna de origem indica a tela/comportamento correspondente. Regras de negócio aparecem em **tabelas de decisão** (determinísticas, para o dev) e em **critérios de aceite BDD/Gherkin** (verificáveis, para o QA).

1 • Visão geral e escopo

WatchUp é um app de **organização de mídias** com três pilares: biblioteca pessoal, progresso episódico e radar de lançamentos. O MVP cobre cadastro rápido de mídias, acompanhamento de progresso por episódio e uma lista de estreias organizada por status de data.

Convenções deste documento

- **IDs de tela** (S001, S006...) seguem o brief. Telas sem ID formal recebem nome descritivo.
- **Campos funcionais** usam os nomes do domínio (`statusLancamentoEpisodico`) — não os rótulos de UI.
- **“Em dia”** é um **estado derivado** de exibição (ver §5.3), não uma escolha do usuário.

Fidelidade do protótipo. O protótipo usa dados em memória (seed local, buscas simuladas com *delay*, salvamento com 1,2 s de latência fake e simulação de erro opcional). Este documento marca esses pontos como **[PROTÓTIPO]** e indica a implementação real esperada no Android.

2 • Arquitetura de navegação

Navegação em **bottom navigation com 4 abas** e um **FAB central** de cadastro. Detalhe, Progresso e Cadastro/Edição são destinos empilhados sobre a aba atual.

Destino	Tipo	Entrada	Back stack
Início (S001)	Aba	Tab bar	Reseta histórico da aba
Lançamentos (S006)	Aba	Tab bar	Reseta histórico da aba
Buscar (S008)	Aba	Tab bar	Reseta histórico da aba
Biblioteca	Aba	Tab bar	Reseta histórico da aba
Cadastro / Edição (S010–S015 / S018)	Push (fluxo)	FAB · CTA vazio · item de busca/lançamento · lápis (editar)	Empilha; “voltar” recua etapa a etapa
Detalhe (S016)	Push	Toque em qualquer card de mídia	Empilha sobre a origem
Progresso (S017)	Push	“Atualizar progresso” no Detalhe	Volta ao Detalhe ao salvar

Regra de back: trocar de aba limpa a pilha; ações internas (abrir detalhe, avançar cadastro) empilham. Salvar cadastro leva ao Detalhe da mídia recém-criada com a pilha reiniciada em Início.

3 • Modelo de dados (contrato Android)

Entidade única **Mídia** persistida via **Room**. Enums abaixo mapeiam 1:1 com os campos do protótipo. **statusMídia** e o estado “Em dia” são **derivados** na camada de domínio (não persistidos como fonte de verdade — ver §5).

3.1 Enums

```
enum class TipoMidia(val episodica: Boolean) {
    FILME(false), DOCUMENTARIO(false), SHOW(false), OUTRO(false),
    SERIE(true), ANIME(true), REALITY(true), PROGRAMA(true)
}

enum class Contexto { SOZINHO, ACOMPANHADO, TANTO_FAZ, NAO_DEFINIDO }
enum class Modalidade { STREAMING, CINEMA, OUTRO, NAO_SEI }
enum class StatusLancEpisodico { COMPLETA, LANCANDO, VAI_LANCAR }
enum class StatusData { DEFINIDA, SEM_DATA, NAO_INFORMADO, NAO_APLICA }

// Derivados (não escolhidos pelo usuário)
enum class StatusMidia { LANCANDO, EM_CARTAZ, EM_BREVE, FINALIZADA }
enum class StatusUsuario { QUERO_ASSISTIR, ASSISTINDO, VISTO } // "Em dia" = derivado
```

3.2 Entidade Room

```
@Entity(tableName = "midia")
data class Midia(
    @PrimaryKey(autoGenerate = true) val id: Long = 0,
    val tipo: TipoMidia,
    val titulo: String,
    val ano: Int? = null,
    val genero: String,
    val contexto: Contexto,

    // Onde assistir (modalidade + detalhe)
    val modalidade: Modalidade,
    val streamings: List<String> = emptyList(), // @TypeConverter (JSON)
    val streamingPrincipal: String? = null,      // ∈ streamings
    val cinemaRede: String? = null,

    // Datas e status de lançamento
    val statusLancEpisodico: StatusLancEpisodico? = null, // só episódica
    val statusData: StatusData,
    val dataPrincipal: LocalDate? = null,                // aceita passado/futuro
    val diaLancamento: String? = null,                  // só episódica LANCANDO
    val horarioLancamento: String? = null,

    // Disponibilidade episódica (≠ progresso)
    val temporadasDisponiveis: Int = 0,
    val temporadaAtual: Int = 0,
    val episodiosDispTempAtual: Int = 0,

    // Progresso do usuário (só ASSISTINDO)
    val ultimoEpisodioVisto: Int = 0,
    val statusUsuario: StatusUsuario,
)

// Episódios por temporada – tabela filha (editável no Detalhe, §6.5)
@Entity(tableName = "episodios_temporada",
    sc-camel-primary-keys = ["midiaId", "temporada"])
data class EpisodiosTemporada(
    val midiaId: Long,
    val temporada: Int,
    val quantidade: Int,
)
```

No protótipo `episodios` é um mapa `{temporada: qtd}`. Em Room, modele como tabela filha (acima) ou `Map<Int,Int>` via `TypeConverter`.

Camadas. Persistência local (Room) para o MVP. Deixe `MidiaRepository` como interface para futura sincronização remota (Retrofit) sem tocar na UI. Estados derivados vivem em *use cases*/domínio, não na Entity nem na tela.

4 • Dicionário de campos

Contrato dos campos capturados no cadastro (S010–S015). “Obrigatório” é condicional — ver a coluna de condição e as tabelas de decisão em §5.

Campo	Tipo Kotlin	Valores / opções	Obrigatório quando
tipo	TipoMidia	filme, série, anime, documentário, show, reality, programa, outro	Sempre (etapa 1)
titulo	String	texto livre; busca em catálogo	Sempre
ano	Int?	ano numérico	Opcional (auto ao escolher do catálogo)
genero	String	texto editável	Sempre
contexto	Contexto	sozinho, com alguém, tanto faz, não definido	Sempre
modalidade	Modalidade	streaming, cinema, outro, não sei ainda	Sempre
streamings	List<String>	Netflix, Prime, Max, Disney+, Apple TV+, Crunchyroll, Globoplay, Paramount+	modalidade = streaming (≥ 1)
streamingPrincipal	String?	um dos streamings selecionados (★)	modalidade = streaming
cinemaRede	String?	Cinemark, UCI, Kinoplex, Cinépolis, Cinesystem, Outro, Não definido	modalidade = cinema
statusLancEpisodico	StatusLancEpisodico?	completa, lançando, vai lançar	tipo.episódica = true
statusData	StatusData	definida, sem data, não informado, não se aplica	Sempre
dataPrincipal	LocalDate?	aceita passado / presente / futuro	statusData = definida
diaLancamento / horario	String?	ex.: “Domingo” / “22:00”	episódica ∧ lançando (opcional)
temporadasDisponiveis	Int	≥ 0	episódica ∧ statusLancEp ≠ vai lançar
temporadaAtual	Int	≥ 1	bloco Progresso visível
episodiosDispTempAtual	Int	≥ 0	bloco Progresso visível
ultimoEpisodioVisto	Int	0 ≤ x ≤ episodiosDispTempAtual	bloco Progresso visível
statusUsuario	StatusUsuario	quero assistir, assistindo, visto	Sempre

5 • Regras de negócio — tabelas de decisão

5.1 Visibilidade condicional do cadastro (etapa 5)

Blocos exibidos na etapa “Datas e status” conforme `tipo`, `statusLancEpisodico` e `statusUsuario`.

Condição de entrada	Status de lançamento	Dia/horário	Disponibilidade	Progresso
Não episódica (filme, doc, show, outro)	—	—	—	—
Episódica · vai lançar	✓	—	—	—
Episódica · completa	✓	—	✓	se Assistindo
Episódica · lançando	✓	✓	✓	se Assistindo

✓ visível · — oculto. O bloco **Progresso** só aparece quando `statusUsuario = ASSISTINDO` e Disponibilidade está visível.

5.2 Derivação de `statusMidia`

Entrada	<code>statusMidia</code> resultante
Episódica · <code>sc-camel-status-lanc-ep</code> = completa	FINALIZADA
Episódica · <code>sc-camel-status-lanc-ep</code> = lançando	LANÇANDO
Episódica · <code>sc-camel-status-lanc-ep</code> = vai lançar	EM BREVE
Não episódica · <code>sc-camel-status-data</code> = definida $\wedge$ data > hoje	EM BREVE
Não episódica · demais casos	EM CARTAZ

5.3 `statusUsuario` e o estado derivado “Em dia”

```
// "Em dia" NÃO é uma opção do usuário — é derivado para exibição.
fun statusExibicao(m: Midia): String = when {
    m.tipo.episodica
        && m.statusUsuario == ASSISTINDO
        && m.episodiosDispTempAtual > 0
        && m.ultimoEpisodioVisto >= m.episodiosDispTempAtual -> "Em dia"
    else -> m.statusUsuario.rotulo()
}
```

Ao salvar progresso, se `ultimoEpisodioVisto ≥ episodiosDispTempAtual` o usuário fica “Em dia”; se cair abaixo de novo, volta a “Assistindo”. Persistir sempre `ASSISTINDO` e derivar “Em dia” em runtime.

5.4 Validações por etapa do cadastro

Etapa	Regra	Mensagem de erro
2 · Título	título não vazio	“Informe um título.”
3 · Detalhes	gênero não vazio	“Informe o gênero.”
3 · Detalhes	contexto selecionado	“Escolha o contexto de consumo.”
4 · Onde assistir	modalidade selecionada	“Selecione onde vai assistir.”
4 · Streaming	≥ 1 streaming	“Selecione ao menos um streaming.”
4 · Streaming	principal definido (★)	“Defina o streaming principal (★).”
4 · Cinema	cinema/rede informado	“Informe o cinema/rede.”
5 · Datas	episódica → statusLancEp selecionado	“Selecione o status de lançamento.”
5 · Datas	statusData selecionado	“Selecione o status da data.”
5 · Datas	statusData = definida → data preenchida (passado permitido)	“Informe a data principal.”
5 · Disponib.	bloco visível → temporadas disp. preenchido	“Obrigatório.”
5 · Status	statusUsuario selecionado	“Escolha seu status.”
5 · Status	episódica ∧ statusLancEp ≠ completa ∧ Visto → bloqueado	“Só marque ‘Visto’ quando a série estiver Completa.”
5 · Progresso	temporada atual / eps disp. / último visto preenchidos	“Obrigatório.”
5 · Progresso	último visto ≤ eps disponíveis	“Não pode ser maior que os episódios disponíveis (N).”

5.5 Opções de statusUsuario por tipo

- **Não episódica:** Quero assistir · Visto.
- **Episódica:** Quero assistir · Assistindo · Visto — com **“Visto” desabilitado** (com cadeado) enquanto `statusLancEp ≠ completa`.

6 · Especificação por tela

S001 Início / Home

Objetivo: ponto de entrada com retomada de conteúdo e radar imediato de estreias.

Componentes: cabeçalho (logo + avatar); carrossel “Continuar assistindo” (mídias episódicas com status Assistindo/Em dia, mostrando barra de progresso e badge); seção “Estreias desta semana” (3 primeiros lançamentos da semana); estado vazio da biblioteca com CTA “+ Adicionar mídia”.

Estados: biblioteca vazia (empty state) · sem itens em andamento (carrossel oculto) · populada.

S006 Lançamentos

Objetivo: radar de estreias e novos episódios agrupado por status de data.

Seções (ordem fixa): “Esta semana” · “Próximas datas” · “Sem data definida”. Cada seção mostra contagem e, se vazia, o texto “Nenhum item neste filtro.”.

Filtros por tipo de data: Todos · Estreia cinema · Estreia streaming · Novo episódio.

Ações do item: toque abre o Detalhe; “+ Salvar” inicia o cadastro pré-preenchido (título, ano, gênero, tipo). Itens sem data exibem “A definir” em cor neutra.

S008 Buscar

Objetivo: localizar uma mídia no catálogo para adicionar.

Estados (obrigatórios): *inicial/vazio* (instrução) · *carregando* (skeleton de 3 linhas) · *sem resultado* (com CTA “Cadastrar manualmente”) · *com resultados* (cada item com “+ Adicionar” → cadastro pré-preenchido).

[PROTÓTIPO] busca no catálogo local com *debounce* de 650 ms. No Android: mesma UX de estados, alimentada por repositório (local ou API).

S00x Biblioteca

Objetivo: acervo pessoal em grade.

Componentes: filtros por tipo (Todos · Filmes · Séries · Animes · Docs — “Séries” agrupa série/programa/reality); grade 3 colunas com pôster placeholder, chip de status do usuário sobreposto, título e tipo.

Estados: vazio (por filtro) com CTA de adicionar · populada.

S016 Detalhe da mídia

Objetivo: visão completa + ponto de partida para progresso e edição.

Cabeçalho: pôster, título, meta (tipo · ano · gênero) e **dois status distintos** — “Status da mídia” (contorno) e “Seu status” (preenchido). Botão de editar (lápiz) abre o cadastro pré-preenchido (S018).

Blocos condicionais: item vindo de Lançamentos e fora da biblioteca → CTA “Adicionar à biblioteca”; episódica “vai lançar” → aviso “sem disponibilidade ou progresso”; episódica com progresso → card de temporada com barra, “Você está em dia!” ou “Faltam N episódio(s)”, aviso de próximo episódio (dia/horário) e botão “Atualizar progresso”; não episódica → botão “Marcar como assistido”.

Ficha: lançamento, onde assistir (modalidade + detalhe), contexto, status da data (+ data formatada).

Episódios por temporada: lista editável (uma linha por temporada disponível) para **complementar a quantidade de episódios após o cadastro**. Oculto para “vai lançar”. Salva no ato.

S017 Atualizar progresso

Objetivo: registrar o último episódio visto (somente Assistindo).

Componentes: stepper –/+ (limitado a 0...episódios disponíveis) e grade de episódios com 3 estados — **Visto** (preenchido), **Disponível** (contorno) e **Bloqueado** (tracejado, episódios ainda não lançados, não clicáveis).

Regras: não é possível ultrapassar o disponível; ao alcançar o disponível exibe badge “Em dia”; “Salvar progresso” retorna ao Detalhe com toast “Progresso atualizado”.

S010–S015 Cadastro em 6 etapas

Objetivo: adicionar mídia com todos os campos funcionais preservados. Barra de progresso e botão “Avançar” fixos; cada etapa valida antes de seguir (§5.4).

1. **Tipo** — grade de 8 tipos (inclui documentário, reality, programa, show, outro).
2. **Título** — busca/entrada manual; escolher resultado preenche tipo/ano/gênero e pula para a etapa 3.
3. **Detalhes** — gênero (texto) + contexto de consumo.
4. **Onde assistir** — modalidade + detalhe: streaming (chips multi-seleção + ★ principal) ou cinema (chip de rede).
5. **Datas e status** — status de lançamento (episódica), status da data, data principal, dia/horário, disponibilidade, seu status e progresso — todos condicionais (§5.1).
6. **Confirmar** — resumo revisável + salvar. Feedback de sucesso (toast) e erro acionável (“Tentar novamente”).

[PROTÓTIPO] salvamento com latência simulada de 1,2 s e checkbox “Simular falha de conexão”. No Android: persistência real + tratamento de erro equivalente.

S018 Edição de mídia

Reusa as 6 etapas do cadastro com os campos pré-preenchidos a partir da mídia existente. Título muda para “Editar mídia”; ao salvar, atualiza o registro (mesmo id), volta ao Detalhe e exibe toast “Alterações salvas”.

7 • Critérios de aceite (BDD / Gherkin)

Cenários verificáveis por fluxo. Rótulos entre aspas são os visíveis na UI; nomes técnicos referenciam o domínio.

FP001 – Cadastrar filme futuro (cinema)

Funcionalidade: Cadastro de filme com estreia futura no cinema

Cenário: Data futura resulta em statusMídia "Em breve"

Dado que escolhi o tipo "Filme"

E informei título "Avatar 3" e gênero "Aventura"

E escolhi modalidade "Cinema" com rede "UCI"

E marquei status da data "Com data definida"

Quando informo a data 19/12/2025 (futura) e confirmo

Então a mídia é salva com sc-camel-status-midia = EM_BREVE

E vejo o toast "Mídia adicionada à sua biblioteca"

Cenário: Datas no passado são aceitas

Dado status da data "Com data definida"

Quando informo a data 01/03/2024 (passada)

Então o avanço é permitido sem erro de validação

Cenário: Data obrigatória quando definida

Dado status da data "Com data definida" e campo de data vazio

Quando toco em "Avançar"

Então vejo "Informe a data principal." e permaneço na etapa

FP002 – Cadastrar série/anime com status de lançamento

Funcionalidade: Visibilidade condicional na etapa de datas/status

Cenário: "Vai lançar" oculta disponibilidade e progresso

Dado tipo episódico e sc-camel-status-lanc-ep = "Vai lançar"

Então os blocos Disponibilidade e Progresso ficam ocultos

E o campo dia/horário fica oculto

Cenário: "Completa" oculta dia/horário de lançamento

Dado tipo episódico e sc-camel-status-lanc-ep = "Completa"

Então dia/horário ficam ocultos

E o bloco Disponibilidade fica visível

Cenário: "Lançando" exibe dia/horário e disponibilidade

Dado tipo episódico e sc-camel-status-lanc-ep = "Lançando"

Então dia/horário, Disponibilidade e Progresso (se Assistindo) ficam visíveis

Cenário: Bloqueio de "Visto" em série lançando

Dado tipo episódico e statusLancEp ≠ "Completa"

Então a opção "Visto" aparece desabilitada com cadeado

E a dica "Requer status Completa" é exibida

FP003 – Atualizar progresso e exibir "Em dia"

Funcionalidade: Progresso episódico

Cenário: Só Assistindo tem progresso

Dado sc-camel-status-usuario = "Quero assistir" OU "Visto"

Então o card de progresso e o botão "Atualizar progresso" não aparecem

Cenário: Alcançar o disponível marca "Em dia"

Dado sc-camel-status-usuario = "Assistindo" e episódios disponíveis = 5

Quando marco o último episódio visto = 5 e salvo

Então o status de exibição passa a "Em dia"

E vejo o toast "Progresso atualizado"

Cenário: Episódios não lançados ficam bloqueados

Dado episódios disponíveis = 5

Então os episódios 6+ aparecem bloqueados (cadeado) e não são clicáveis

E o stepper "+" não passa de 5

FP004 – Lançamentos por status de data

Funcionalidade: Radar de lançamentos

Cenário: Agrupamento em três seções

Quando abro "Lançamentos"

Então vejo "Esta semana", "Próximas datas" e "Sem data definida" com contagem

Cenário: Itens sem data exibem "A definir"

Dado um item do grupo "Sem data definida"

Então sua data aparece como "A definir" em cor neutra

Cenário: Filtro por tipo de data

Quando seleciono o filtro "Estreia cinema"

Então só itens de cinema permanecem; seções vazias mostram

"Nenhum item neste filtro."

FP005 – Cadastro completo de metadados
Funcionalidade: Campos obrigatórios não colapsam

Cenário: Todos os campos-chave são capturados
Dado que percorro o cadastro (S011–S014)
Então informo tipoMídia, genero, contexto e onde assistir
E para streaming defino ao menos um serviço e o principal (*)
E para cinema informo a rede
E nenhum desses dados é omitido no resumo de confirmação

FP006 – Episódios por temporada no detalhe
Cenário: Complementar episódios após o cadastro
Dado uma série episódica na biblioteca (não "vai lançar")
Quando edito a quantidade de episódios de uma temporada no Detalhe
Então o valor é salvo e refletido na estrutura por temporada

Estados globais – Erro de salvamento acionável
Cenário: Falha ao salvar permite nova tentativa
Dado que a gravação falha
Então vejo "Não foi possível salvar" com ação "Tentar novamente"
E ao repetir com sucesso a mídia é adicionada e o Detalhe abre

8 • Estados globais e feedback

Padrão	Onde	Comportamento
Vazio	Home, Biblioteca, Busca	Ilustração/placeholder + mensagem + CTA de ação
Carregando	Busca, Salvar	Skeleton (busca) · spinner no botão (salvar)
Erro	Salvar, validações	Bloco de erro acionável (retry) · erros inline por campo
Sucesso	Salvar, progresso, marcar visto, editar	Toast (≈2,4 s): “Mídia adicionada...”, “Progresso atualizado”, “Marcado como assistido”, “Alterações salvas”

9 • Matriz de rastreabilidade

Fluxo	Telas	Regras (§)	Aceite
FP001 Filme futuro	S010–S015	5.2, 5.4	§7 • FP001
FP002 Série lançando	S010–S015	5.1, 5.5	§7 • FP002
FP003 Progresso	S016, S017	5.3	§7 • FP003
FP004 Lançamentos	S006	—	§7 • FP004
FP005 Metadados	S011–S014	4, 5.4	§7 • FP005
FP006 Eps/temporada	S016	3.2	§7 • FP006

10 • Decisões abertas

- **Persistência de dados:** confirmar Room local-only no MVP vs. sincronização remota (definir contrato de API se sim).
- **Origem do catálogo de busca:** catálogo próprio ou integração externa (TMDB etc.) — impacta o modelo de metadados.
- **Notificações:** lançamentos e novos episódios sugerem push — fora do escopo do protótipo; definir se entra no MVP.
- **Cobertura de tablet** (pendência do brief) — layout responsivo além do phone.
- **Persistência do “Em dia”:** recomendação de derivar em runtime (§5.3) em vez de persistir, para evitar dessincronização.